Integrating PLC Logic Languages with SAST, DAST, IAST, RASP & Threat Modeling

Where it began

In 2018, while working at LEO Cybersecurity, I created the original PLC secure coding practices (https://icubedsolutionsllc.com/blog/f/secure-plc-programming-standards-in-projects-organizations) after being frustrated that we weren't addressing them in part 4-1 of ISA/IEC 62443 which focuses on secure and mature development lifecycle of ICS OT products. With a few contributions from my LEO Cybersecurity teammate Brian Foster who served as my first PLC programmer reviewer, I created 10 recommended practices specific to each of the 5 IEC 61131-3 PLC languages of Ladder Diagram/Ladder Logic (LD), Function Block Diagram (FBD), Sequential Function Chart (SFC), Instruction List (IL) and Structured Text (ST). It was Jim Gilsinn and Clint Bodungen who first encouraged me to funnel my frustration into just going ahead and writing down my ideas and then publishing them. Insert the angry black man jokes here for my friends in the community haha I pitty da fool that tries to stop me 😊 .

In 2019 I ended up at Jacobs Engineering finally getting to work side by side with an ICS OT community legend and leader, my teammate, Jake Brodsky. Soon as I shared my practices with Jake and started pushing it as a service that I think our ICS OT Security team should provide throughout Jacobs to all our systems integration, instrumentation and control engineering, design and construction teams. Jake realized this was something that folks had not previously really focused on. He also knew the weren't just going to list to a security guy who originated on the IT side years ago. We pushed for Jake to get a chance to talk about this at S4x20 https://www.youtube.com/watch?v=5unyhZCdqLo. The idea wasn't accepted at first by S4 but with some pushing, (we're not going to mention the crazy emails I sent to Dale Peterson and Jake lol), Jake's talk was accepted and to both Jake's and my surprise it went viral overnight. I knew then that what was a dud when I created the ideas and published them in 2018 was now going to get attention and I am forever grateful to my buddy Jake for taking me seriously and using his decades of experience and his name to make the community pay attention. That's something I knew wasn't going to happen with me pushing it alone. But yes the idea originated with me, this Black American Man in 2018 simply out of frustration of why it wasn't being focused on. PLC I and PLC IV from Paul Lynn on Udemy in 2018 with Jim and Clint pushing me to publish my ideas, Jake believing it was worth taking on in 2019 was the only reason this stuff saw the light of day.

Fast forward to post S4x20 and Dale Peterson, Sarah Fluchs, Vivek Ponnada worked with ISA GCA to create a community portal to create the first ever PLC coding practice project for the community. They started with Jake's ideas and experiences, and I submitted my original 2018 practices via Jake as well. We then recruited a global community of PLC programmers, control and instrumentation engineers and technicians as well as security professionals focused on ICS OT security, secured SDLC etcetera. Thanks to Sarah and Vivek and the community of volunteers that project became the Top 20 PLC Secure Coding practices https://plc-security.com/. To add icing on the cake we had PLC OEMs like Bedrock Automation, who built the first ever secure by design PLC, immediately adopt, endorse and encourage their customers to using the Top 20 PLC secure coding practices https://bedrockautomation.com/best-practices-for-cyber-secure-plc-programming/.

Thanks to Sarah and Vivek's leadership my original 2018 spark that became Jake's S4x20 flame has now become a 2021 fire that will be sustained by the community for years to come. For that I am grateful and continue to be amazed. It may have taken years, it may have taken me pushing this to others who don't look like me but the idea and some of the practices live on through the community. Crazy ideas sometimes take years for everyone to think it's worth it or relevant but when they do eventually get picked up in the wind and from the currents it's a beautiful thing to watch what they become.

Existing Secure Coding Methodologies

With all that history aside and my crazy ideas sparking something much bigger than me, let me begin with more crazy ideas that will once again hopefully catch fire within the community. The traditional IT secure coding community already has methodologies with products and tools behind them. The ones I want to focus on are:

- **Static Application Security Testing (SAST) –** this is using manual or automated code check tools to review raw static code as developers are creating it. It enables and empowers developers to look for commonly known issues like input validation, buffer issues etcetera in their initial code.
- **Dynamic Application Security Testing (DAST) –** this is usually simulating an attacker attempting to dynamically attack, fuzz and manipulate already compiled code and applications without having the raw static original source code from the developers. Usually tools like Burp Suite, Web Scarab, w3af and others are used for DAST on applications.
- **Interactive Application Security Testing (IAST) –** according to Veracode IAST is "analyzes code for security vulnerabilities while the app is run by an automated test, human tester, or any activity "interacting" with the application functionality." In plain English, this is capturing or simulating real-time user interactions with the application to find weaknesses that may be missed by SAST and DAST approaches.
- **Runtime Application Self-Protection (RASP) –** this runs on an production application server enabling the applications and servers to protecting themselves in real-time production environments from known attacks by monitoring real-time behaviors of the application within it's production context and the expected production inputs being crossed analyzed within the relevant behavioral context.
- **Threat Modeling –** a process and methodology that enables you to identify potential threats, weaknesses, vulnerabilities, design gaps, impacts, consequences etcetera specific to the systems, devices, applications and their expected usage context and environments.

Each of these approaches now have industry tools made by companies such as Veracode, Imperva, Synopsis, Threatmodeler, Avocado and others so that traditional programmers and development lifecycles can scale secure coding practices throughout their development lifecycle. However, we currently lack such PLC OEM neutral tools. So why not have those companies doing it for IT applications SDLC today begin to learn the IEC 61131-3 PLC logic languages and work with the PLC OEMs to create tools for reading the PLC logic languages?

Vision, Recommendations & Requests for Tomorrow

So getting to the point of the this article in the first place, I have more random crazy ideas. Now that we have official community led and supported and voted Top 20 PLC secure coding practices let's push

forward with continuing to advance the ICS OT community. My requests, recommendations and vision for the future includes:

- Sarah and Vivek and the Top 20 project to continue the great work they're doing especially with translating the Top 20 into many languages
- The Top 20 project opens a wiki section on their page following the example of OWASP and show all the other PLC logic related practices with a voting poll for picking the next Top 20 practices for 2022 and beyond. Ensure it maintains the Top 20 that were voted from previous years as well as the community continues to keep the fire alive. This way the community can leverage all of the practices from year to year and see the full list of practices that didn't make the version 1 list.
- Top 20 project continue with plans to publish the Top 20 PLC coding environment practices. These were practices that were needed but did not directly involve programming the PLC logic languages themselves. However, they were equally important in enabling the ICS OT community to protect the integrity of PLC programming practices and ICS OT operations in general.
- PLC OEMs, like Bedrock Automation, to actively publish and include Top 20 PLC secure coding practices as well as my original 2018 practices for each of the IEC 61131-3 PLC logic languages into their PLC coding platforms. Not just include them but document them and follow Bedrock's example of encouraging their customers to practice using them.
- Existing secure coding vendors like Veracode, Avocado, Imperva, Synopsis, Threatmodeler etcetera to learn the IEC 61131-3 PLC logic languages and work with the Top 20 PLC coding practices project as well as PLC OEMs to begin adding these practices to their SAST, DAST, IAST, RASP and Threat Modeling tools and capabilities so that the ICS OT community can now benefit from the scaling abilities of those existing automated secure development lifecycle products and methodologies within PLC logic languages
- All regulated industries as well as ICS OT standards and certification orgs adopt the PLC secure coding practices into their existing regimes so that they become mandatory due diligence and due care practices. Do not create new practices, let that stay with the community of volunteers but as the global community of volunteers creates these good practices the standards, certification and regulatory regimes should use basic sense and do what I originally wanted in 2018 and include these practices into their existing GMP, SDLC requirements.
- All secure coding and PLC programming organizations, academic institutions and programming trainers' etcetera must include at a minimum the Top 20 PLC secure coding practices into their existing training as a training module on secure and safe PLC coding. Add this as a module to the existing secure coding certifications as well. All PLC programmers should learn safe and secure practices. All secure coders should learn the PLC IEC 61131-3 logic languages and accept that they are the programming and "developer" languages in the ICS OT world, in the physical world. Make it part of the curriculum today and forever.
- OWASP community to learn the IEC 61131-3 languages and join the Top 20 project to add their many moons of experiences to help Sarah, Vivek and the ICS OT community of volunteers to grow the PLC coding practices to the level of maturity that OWASP has managed to lead and become for more than a decade. We as a society and an ICS OT community need this to protect the physical world and OWASP has the experience to assist the ICS OT community via the Top 20 project.

Conclusion

Just like in 2018, I'm sure my ideas will not initially be understood by the community for some years. However, thanks to Jake, Sarah, Vivek and others I now have faith that regardless of how many years it takes, eventually that spark will become a fire that then becomes a flame. The US Navy and Penn State University both taught me long ago that sometimes it's best to put the message in a bottle and put it in the water with a little sail and let it go than it is to try to predict or control the flow of the currents and the direction of the wind. Many thanks to Sarah and Vivek especially for keeping the fire going and keeping the community of volunteers focused on PLC coding practices alive and well for years to come. I will as always continue to sail the seas wherever the currents and the winds take me. It is good to see we finally did something with immediate impact that didn't require anyone to buy any security products to do it out the gate. We empowered everyday folks to do something now not tomorrow and now with needing to request budget. The more we can create ideas like that the more I have faith we'll eventually reach the promised land if there ever was such a thing for ICS OT security and safety. Never out of the fight. Cheers. Isiah Jones, the Black American ICS OT cyber maverick, out.